Chapter 23: State Management

1. Overview

Every non-trivial Angular application accumulates state that outlives a single component: the logged-in user, a shopping cart, filters on a data grid, feature flags, in-flight HTTP requests, cached API responses, wizard-step progress, websocket messages. The moment two unrelated components need to read or react to the same piece of data, you have a state management problem, whether you admit it or not.

"State management" is not synonymous with "NgRx." NgRx is one library-shaped answer to a much older question: **where does shared, mutable-over-time data live, who is allowed to change it, and how do interested parties find out when it changes?** Angular gives you the raw materials to answer that question yourself (classes, `Subject` / `BehaviorSubject`, and now `signal`), and a spectrum of libraries (NgRx, NgRx Signal Store, Akita, Elf, plain RxJS stores) that each pick a different set of trade-offs along that spectrum.

This chapter is deliberately conceptual and comparative, not a full NgRx tutorial (a dedicated NgRx scenario chapter elsewhere goes deep on that). The goal here is that you can, in an interview or in a design review, articulate:

- Why state management is a real engineering problem, not resume-driven complexity.
- The full spectrum of solutions from "a service with a `BehaviorSubject`" to "a service with signals" to "full NgRx" to "NgRx Signal Store," and what each buys you.
- The core theoretical pillars underneath all of them: unidirectional data flow and immutability.
- A decision framework for picking the right tool for a given app size and team.

2. Core Concepts

2.1 Why state management is needed

In a naive Angular app, state lives wherever it's convenient: a component's field, a service's property, a value stashed in `localStorage`. This works until:

1. **Multiple components need the same data.** Two sibling components (not parent/child) both need the current user's cart. Passing it via `@Input()` / `@Output()` chains ("prop drilling") through components that don't care about the data becomes unmaintainable as the tree grows.
2. **State changes need to be observed reactively.** A component doesn't just need the cart once — it needs to re-render whenever the cart changes, even if the change was triggered by a completely different part of the UI.
3. **State outlives the component that created it.** Navigate away from a page and back — should the filters you set still be applied? If yes, that state cannot live in the component instance (which gets destroyed on navigation).
4. **Change causes are hard to trace.** When any component can mutate a shared object directly, a bug ("why is the cart total wrong?") requires searching the entire codebase for every place that touches `cart`. There's no single choke point to add a breakpoint, a log, or a guard.
5. **Consistency across async operations.** An HTTP call updates the server; meanwhile the UI has already optimistically changed; a websocket push changes it again. Without a disciplined single source of truth, these three writers race and produce inconsistent UI.

6. **Testability and time-travel/debugging.** Ad-hoc mutable state scattered across services is hard to unit test in isolation and impossible to "replay." A single, serializable state tree with pure transition functions can be trivially tested, logged, serialized, and replayed (this is exactly what NgRx Store gives you, and why Redux-style architectures became popular for debugging tools like time-travel).

The common thread: as an app grows, you need a **single, predictable source of truth**, a **defined way to change it**, and a **defined way to observe it**. Everything in this chapter is a different implementation of those three requirements.

2.2 The spectrum of solutions

Think of state management as a ladder. Climb only as high as your app's complexity demands — every rung adds power but also ceremony and indirection.

```
Rung 0: Component-local state (fields, no sharing)
Rung 1: A shared service holding plain properties (no reactivity)
Rung 2: A service with Subjects/BehaviorSubjects (RxJS-based reactive store)
Rung 3: A service with signals (signal-based reactive store)
Rung 4: A lightweight state library (Akita, Elf, or a hand-rolled signal store)
Rung 5: NgRx Signal Store (structured, DI-based signal store with features)
Rung 6: Full NgRx (Store + Actions + Reducers + Effects + Selectors)
```

Rung 1 — Plain shared service

```
@Injectable({ providedIn: 'root' })
export class CartService {
  items: CartItem[] = [];
  addItem(item: CartItem) { this.items.push(item); }
}
```

This "works" because Angular's DI gives every consumer the *same instance*. But there is no notification mechanism — a component that isn't the one calling `addItem()` has no way to know the array changed unless it polls or you wire up manual events. Fine for truly trivial, single-consumer state; wrong the moment two independent parts of the UI must stay in sync.

Rung 2 — Service with Subjects (the classic "poor man's store")

```
@Injectable({ providedIn: 'root' })
export class CartService {
  private itemsSubject = new BehaviorSubject<CartItem[]>([]);
  readonly items$ = this.itemsSubject.asObservable();

  addItem(item: CartItem) {
    const current = this.itemsSubject.value;
    this.itemsSubject.next([...current, item]); // immutable update
  }
}
```

This is the pattern every Angular developer should be fluent in before reaching for a library. It gives you:

- A single source of truth (`itemsSubject`).

- Reactive notification (`items$`, consumed with `async` pipe or `toSignal`).
- An encapsulated write API (only `addItem`/similar methods can mutate; the raw subject is private).

It does **not** give you: enforced immutability (nothing stops you from doing `this.itemsSubject.value.push(item)` by mistake), a standard way to compose derived state, action logging/replay, or a convention for handling async side effects consistently across a large team. This pattern scales surprisingly far for small-to-medium apps and is often what interviewers want to hear you can build **before** they ask about NgRx.

Rung 3 — Service with signals

Since Angular 16+ signals, the same pattern is expressible without RxJS boilerplate, and it composes beautifully with `computed()` for derived state and Angular's change detection (signals notify exactly the consumers that read them, enabling fine-grained reactivity and, eventually, zoneless change detection).

```
@Injectable({ providedIn: 'root' })
export class CartService {
  private readonly _items = signal<CartItem[]>([]);
  readonly items = this._items.asReadonly();
  readonly total = computed(() =>
    this._items().reduce((sum, i) => sum + i.price * i.qty, 0)
  );

  addItem(item: CartItem) {
    this._items.update(items => [...items, item]); // immutable update
  }
}
```

This is now the idiomatic "simple state" answer in modern Angular (v16+): no subscriptions to manage, no `async` pipe needed in templates, automatic unsubscription (signals have no subscription to leak), and derived state via `computed()` is memoized for free. For many apps this rung is the sweet spot and you never need to go further.

Rung 4/5 — Lightweight libraries and NgRx Signal Store

Once you have several such stores, you start wanting **conventions**: consistent naming, computed selectors, method organization, testing helpers, devtools integration, and features like entity collections (normalized lists with CRUD helpers). This is what Akita, Elf, and NgRx Signal Store provide — structure around the "service with signals/subjects" pattern rather than a fundamentally different architecture. NgRx Signal Store, specifically, is built on the `signalStore()` function with composable "features" (`withState`, `withComputed`, `withMethods`, `withHooks`, `withEntities`) — see §3.3.

Rung 6 — Full NgRx (Redux pattern)

NgRx adds the strict, unidirectional Redux architecture on top of RxJS: **Store** (single immutable state tree), **Actions** (typed events describing "what happened," not "what to do"), **Reducers** (pure functions computing new state from old state + action), **Selectors** (memoized derived-state queries), and **Effects** (isolate side effects like HTTP calls, listening for actions and dispatching new ones). This buys you: enforced unidirectional flow (nothing can mutate state except a reducer, and reducers must be pure), excellent devtools (time-travel debugging, action replay, state diffing), strong testability (reducers/selectors are pure functions, effects are testable in isolation with marble testing), and a consistent pattern across a large team/codebase. The cost:

significant boilerplate (even with the modern `createActionGroup` / `createReducer` /functional-effects APIs it's still more ceremony than a signal), a learning curve (RxJS operators, the action/reducer/effect mental model), and indirection (a single button click can fan out across 4 files before you see the state change).

2.3 Decision framework

Ask, in order:

1. **Is the state used by only one component subtree (parent + its children)?** → Component `@Input()` / `@Output()` or a component-scoped service (provided at the component level, not root). No global store needed.
2. **Is it shared across unrelated parts of the app, but the app is small/medium, has a small team, and doesn't need time-travel debugging or heavy async orchestration?** → A service with signals (or `BehaviorSubject` if you're pre-Angular-16 or the state is fed by RxJS streams like websockets/polling). This is correct for the majority of line-of-business apps.
3. **Do you need structured entity management (normalized collections), computed state, or just want conventions across many such services without full Redux ceremony?** → NgRx Signal Store, Elf, or Akita.
4. **Do you have complex, interdependent async workflows (e.g., action A must trigger B and C, with retries/cancellation/race conditions), a large team that benefits from an enforced convention, or a hard requirement for devtools time-travel/audit-log style debugging?** → Full NgRx.
5. **Is the "state" actually just server data you fetch, cache, and refetch (not really *client* state)?** → Consider a server-state library concern (e.g., NgRx's `@ngrx/entity`, or a resource/cache-focused tool) rather than modeling it as hand-rolled global client state at all — the problems (caching, dedup, invalidation, background refetch) are different from client state problems.

The overarching principle interviewers want to hear: **state management complexity should be proportional to state complexity, not to what's fashionable.** Introducing full NgRx for an app with three shared booleans is over-engineering; refusing to introduce any structure in a 200-component enterprise app with a 20-person team is under-engineering. Also worth mentioning: these approaches are not mutually exclusive within one app — it's common to use NgRx for a few genuinely complex, cross-cutting domains (auth, cart, real-time order status) while everything else uses simple signal-based services.

2.4 Unidirectional data flow

Regardless of which rung you're on, the healthiest architectures share one shape:

```
View (template) --dispatches event--> Action/Method
Action/Method --triggers--> State transition (reducer / update())
State transition --produces--> New state
New state --notifies--> View re-renders
```

Data flows in one direction around this loop. The view never mutates state directly; it only requests a transition. This is the opposite of two-way binding on shared mutable objects, where any consumer can write to the object at any time and there's no single place to intercept or audit the write. Unidirectional flow is what makes state changes traceable: if you see a wrong value in the UI, you can trace it back through exactly one path (view → action → transition function → new state) instead of searching for every possible writer.

2.5 Immutability

Every pattern in this chapter (signals, `BehaviorSubject`-based stores, and NgRx reducers) depends on **producing new object/array references on every change** rather than mutating in place. Two reasons this matters:

1. **Change detection / memoization correctness.** Angular's `OnPush` change detection, `computed()` signals, and NgRx's `createSelector` all use reference equality (`===`) to decide "did this actually change?" If you mutate an array in place (`items.push(x)`) and pass the *same reference* back, downstream consumers doing `oldRef === newRef` see no change and skip re-rendering/recomputing — a silent, hard-to-diagnose bug.
2. **Predictability and debuggability.** If state can never be mutated except by producing a new object through a defined transition function, then every past state is a frozen snapshot. This is what enables time-travel debugging in NgRx DevTools and makes reasoning about "what was the state when this bug happened" possible.

Practically, immutable updates use spread syntax or array methods that return new arrays (`map`, `filter`, `concat`) instead of mutating ones (`push`, `splice`, `sort` in place), e.g. `this._items.update(items => [...items, newItem])` rather than `this._items().push(newItem)`.

3. Code Examples

3.1 A signal-based state store built from scratch

```
// todo-store.service.ts
import { Injectable, signal, computed } from '@angular/core';

export interface Todo {
  id: number;
  text: string;
  completed: boolean;
}

type Filter = 'all' | 'active' | 'completed';

@Injectable({ providedIn: 'root' })
export class TodoStore {
  // Private writable signals — the only place mutation happens.
  private readonly _todos = signal<Todo[]>([]);
  private readonly _filter = signal<Filter>('all');
  private nextId = 0;

  // Public read-only views — consumers cannot call .set()/.update() on these.
  readonly todos = this._todos.asReadonly();
  readonly filter = this._filter.asReadonly();

  // Derived/computed state — memoized automatically, recomputed only
  // when _todos or _filter actually change.
  readonly filteredTodos = computed(() => {
    const todos = this._todos();
    switch (this._filter()) {
```

```
      case 'active':    return todos.filter(t => !t.completed);
      case 'completed': return todos.filter(t => t.completed);
      default:          return todos;
    }
  });

  readonly activeCount = computed(
    () => this._todos().filter(t => !t.completed).length
  );

  // Public API — the only way to mutate state ("actions" in spirit).
  add(text: string): void {
    this._todos.update(todos => [
      ...todos,
      { id: this.nextId++, text, completed: false },
    ]);
  }

  toggle(id: number): void {
    this._todos.update(todos =>
      todos.map(t => (t.id === id ? { ...t, completed: !t.completed } : t))
    );
  }

  remove(id: number): void {
    this._todos.update(todos => todos.filter(t => t.id !== id));
  }

  setFilter(filter: Filter): void {
    this._filter.set(filter);
  }
}
```

```
// todo-list.component.ts
@Component({
  selector: 'app-todo-list',
  standalone: true,
  template: `
    <p>{{ store.activeCount() }} items left</p>
    <ul>
      @for (todo of store.filteredTodos(); track todo.id) {
        <li [class.done]="todo.completed" (click)="store.toggle(todo.id)">
          {{ todo.text }}
        </li>
      }
    </ul>
  `,
})
export class TodoListComponent {
  protected readonly store = inject(TodoStore);
}
```

Notice the shape: private writable signals, public readonly signals, `computed()` for derived state, and a set of named methods as the only mutation surface — this is unidirectional flow and immutability enforced entirely by convention and TypeScript access modifiers, no library required.

3.2 A minimal NgRx store/reducer/selector/effect example

```
// todo.actions.ts
import { createActionGroup, emptyProps, props } from '@ngrx/store';

export const TodoActions = createActionGroup({
  source: 'Todo',
  events: {
    'Add Requested': props<{ text: string }>(),
    'Add Success': props<{ todo: Todo }>(),
    'Add Failure': props<{ error: string }>(),
    'Toggle': props<{ id: number }>(),
  },
});
```

```
// todo.reducer.ts
import { createReducer, on } from '@ngrx/store';

export interface TodoState {
  todos: Todo[];
  loading: boolean;
  error: string | null;
}

const initialState: TodoState = { todos: [], loading: false, error: null };

export const todoReducer = createReducer(
  initialState,
  on(TodoActions.addRequested, (state) => ({ ...state, loading: true, error: null })),
  on(TodoActions.addSuccess, (state, { todo }) => ({
    ...state,
    loading: false,
    todos: [...state.todos, todo], // new array — never state.todos.push(todo)
  })),
  on(TodoActions.addFailure, (state, { error }) => ({ ...state, loading: false, error })),
  on(TodoActions.toggle, (state, { id }) => ({
    ...state,
    todos: state.todos.map(t => (t.id === id ? { ...t, completed: !t.completed } : t)),
  })),
);
```

```
// todo.selectors.ts
import { createFeatureSelector, createSelector } from '@ngrx/store';

export const selectTodoState = createFeatureSelector<TodoState>('todos');

export const selectAllTodos = createSelector(
```

```
  selectTodoState,
  (state) => state.todos,
);

// Memoized: only recomputes when selectAllTodos's *result reference* changes.
export const selectActiveCount = createSelector(
  selectAllTodos,
  (todos) => todos.filter(t => !t.completed).length,
);
```

```
// todo.effects.ts
import { inject } from '@angular/core';
import { Actions, createEffect, ofType } from '@ngrx/effects';
import { catchError, map, mergeMap, of } from 'rxjs';

export const addTodo$ = createEffect(
  (actions$ = inject(Actions), api = inject(TodoApiService)) =>
    actions$.pipe(
      ofType(TodoActions.addRequested),
      mergeMap(({ text }) =>
        api.create(text).pipe(
          map((todo) => TodoActions.addSuccess({ todo })),
          catchError((err) => of(TodoActions.addFailure({ error: err.message }))),
        ),
      ),
    ),
  { functional: true },
);
```

```
// todo-list.component.ts
@Component({ /* ... */ })
export class TodoListComponent {
  private store = inject(Store);

  todos = this.store.selectSignal(selectAllTodos);
  activeCount = this.store.selectSignal(selectActiveCount);

  add(text: string) {
    this.store.dispatch(TodoActions.addRequested({ text }));
  }
}
```

Flow for one click: component calls `dispatch(addRequested)` → reducer sets `loading: true` (synchronous, immediate) → the effect, listening for `addRequested`, calls the API → on response, dispatches `addSuccess` / `addFailure` → reducer produces the final immutable state → selectors recompute → component re-renders. The component never touches HTTP or reducer logic directly — it only dispatches an action describing intent.

3.3 NgRx Signal Store example

```
```

```
import { signalStore, withState, withComputed, withMethods, patchState } from
'@ngrx/signals';
import { computed, inject } from '@angular/core';

interface TodoState {
  todos: Todo[];
  filter: 'all' | 'active' | 'completed';
}

export const TodoStore = signalStore(
  { providedIn: 'root' },
  withState<TodoState>({ todos: [], filter: 'all' }),

  withComputed(({ todos, filter }) => ({
    filteredTodos: computed(() => {
      const list = todos();
      const f = filter();
      if (f === 'active') return list.filter(t => !t.completed);
      if (f === 'completed') return list.filter(t => t.completed);
      return list;
    }),
    activeCount: computed(() => todos().filter(t => !t.completed).length),
  })),

  withMethods((store, api = inject(TodoApiService)) => ({
    add(text: string) {
      const todo: Todo = { id: Date.now(), text, completed: false };
      patchState(store, (state) => ({ todos: [...state.todos, todo] }));
    },
    toggle(id: number) {
      patchState(store, (state) => ({
        todos: state.todos.map(t => (t.id === id ? { ...t, completed: !t.completed } : t)),
      }));
    },
    setFilter(filter: TodoState['filter']) {
      patchState(store, { filter });
    },
    async loadFromServer() {
      const todos = await api.fetchAll();
      patchState(store, { todos });
    },
  })),
);
```

```
@Component({
  selector: 'app-todo-list',
  standalone: true,
  providers: [TodoStore],
  template: `
    <p>{{ store.activeCount() }} left</p>
    @for (t of store.filteredTodos(); track t.id) {
      <div (click)="store.toggle(t.id)">{{ t.text }}</div>
```

```
    }
  `,
})
export class TodoListComponent {
  protected readonly store = inject(TodoStore);
}
```

NgRx Signal Store gives you the *conventions* of a store (DI-scoped, composable `withState` / `withComputed` / `withMethods` / `withHooks` features, `patchState` for guarded immutable updates) without the Actions/Reducers/Effects ceremony of classic NgRx — it sits between "hand-rolled signal service" and "full NgRx" on the ladder in §2.2.

4. Internal Working

4.1 Unidirectional data flow, mechanically

At runtime, "unidirectional flow" is not a diagram — it's an actual call graph enforced by types and encapsulation:

- The template can only call methods exposed by the component (`(click)="add()"`), never touch a private signal or the reducer directly.
- The component can only call the store's public API (`store.add()` / `dispatch(action)`), never construct a new state object itself.
- The store's transition logic (an `update()` callback, or a reducer function) is the *only* code path that produces a new state reference.
- Every consumer of state (`computed()` , a selector, a template binding) only ever reads; it holds a reference to the *readonly* signal/observable, not the writable one.

This one-way arrow (view → intent → transition → new state → view) is what makes a bug like "the cart total is wrong" tractable: there is exactly one function that can have produced the wrong total, and you put a breakpoint there.

4.2 NgRx's Store as a specialized BehaviorSubject

`@ngrx/store` 's `Store` class literally extends RxJS's `Observable` and internally wraps a `BehaviorSubject` -like mechanism (a `ReplaySubject` /state controller depending on version) that holds the *entire application state tree* as its single value. Simplified model of what happens under the hood:

```
class StoreLike {
  private stateSubject = new BehaviorSubject<AppState>(initialState);

  dispatch(action: Action) {
    const currentState = this.stateSubject.value;
    const newState = rootReducer(currentState, action); // pure function call
    this.stateSubject.next(newState); // push the new immutable tree
  }

  select<T>(selector: (state: AppState) => T): Observable<T> {
    return this.stateSubject.pipe(map(selector), distinctUntilChanged());
  }
}
```

Key points an interviewer wants to hear:

- `dispatch()` is synchronous: it looks up the current value, runs the (combined) root reducer — which fans out the action to every feature reducer registered via `StoreModule.forFeature` / `provideState` — and pushes the single new state object.
- The **root reducer** is a combination (via `combineReducers`) of all the individual feature reducers; each feature reducer only owns and returns its own slice, and the root assembles the slices back into one object.
- Every subscriber (every `select()` / `selectSignal()` call) receives the *same* new state object reference and re-derives its own slice via the selector function, using `distinctUntilChanged` (or signals' equality check) so that unrelated selectors don't re-fire if their specific slice didn't change reference.
- Because it's fundamentally a subject-based push mechanism, everything you know about `BehaviorSubject` (synchronous replay of the latest value to late subscribers, no "memory" beyond the latest value) applies to how the Store behaves.

4.3 How `createSelector` memoizes

```
export function createSelector(...selectors, projector) {
  let lastArgs: any[] | null = null;
  let lastResult: any;

  return (state) => {
    const args = selectors.map(s => s(state));       // 1. run parent selectors
    if (lastArgs && args.every((a, i) => a === lastArgs[i])) {
      return lastResult;                              // 2. inputs unchanged → cached result
    }
    lastArgs = args;
    lastResult = projector(...args);                  // 3. recompute
    return lastResult;
  };
}
```

This is exactly the same idea as Angular's `computed()`: a selector memoizes based on **reference equality of its direct inputs**, not deep equality and not "does the whole state object differ." That's why `selectActiveCount` in §3.2 only recomputes when `selectAllTodos`'s *returned array reference* changes — if some unrelated part of the state tree changes (say, `loading`), `selectAllTodos` returns the *same* `todos` array reference as before (because the reducer for `toggle`/`add` produces a fresh `todos` array only when todos actually change), so `selectActiveCount`'s cache is still valid and it skips recomputation entirely. This cascades: a selector tree only recomputes the branches whose upstream slice reference actually changed — this is the entire performance argument for immutability plus memoized selectors.

5. Edge Cases & Gotchas

Over-engineering small apps with NgRx. Introducing the full Store/Actions/Reducers/Effects stack for an app with two or three shared booleans multiplies file count (4-5 files per feature slice) and onboarding cost for no payoff — there's no complex async orchestration or audit-trail requirement to justify it. A common interview red flag is a candidate reaching for NgRx by default rather than by requirement. The fix is always the decision framework in §2.3: start at the lowest rung that solves the actual problem and only climb when you hit a concrete pain point (untraceable bugs, need for devtools replay, complex effect chains).

Mutating state directly. In a `BehaviorSubject` or signal store, doing `this._items().push(x)` (or `state.todos.push(x)` inside a "reducer") mutates the array in place and returns the *same reference*. Anything relying on reference equality — `OnPush` change detection, `computed()`, `createSelector`, `distinctUntilChanged` — will conclude "nothing changed" and silently fail to update the UI, even though the underlying array now technically has the new item. This is one of the most common and hardest-to-spot NgRx/signal bugs because the data *is* correct in memory; only the notification is missing. The fix is always to produce a new container (`[...arr, x]`, `{...obj, key: val}`) on every change, and to enable `Object.freeze` on state in development (NgRx's `runtimeChecks: { strictStateImmutability: true }`) to catch violations early.

Selector memoization breaking due to new object references. A selector/computed that does `todos.filter(...)` or `todos.map(...)` *inside the projector* produces a brand-new array reference on every single call, even if the filtered contents are logically identical to last time. If that selector's output then feeds into `*ngFor`/`@for` without a proper `track` expression, or into another `computed()` that does a naive reference check, you get either unnecessary re-renders or (subtly) an infinite `computed` recalculation cascade. The fix: track a stable id (`track todo.id`) in templates, and be deliberate about which layer owns "compute once" (a memoized selector) vs. "recompute every render" (an unmemoized `.filter()` call in a template getter — a classic anti-pattern to flag in review).

Effects causing infinite dispatch loops. An `Effect` that listens for action `A` and, in its error/success handler, dispatches action `A` again (directly or via a chain: `A` → `B` → `C` → `A`) creates an infinite loop that pegs the CPU and floods memory/devtools. This commonly happens by accident when a generic "refresh" effect is added that listens too broadly (e.g., `ofType(TodoActions.addSuccess, TodoActions.toggleSuccess, ...)` and both dispatches and is itself triggered by an action in that same list). Guard against it by: keeping each effect's listened-action-set and dispatched-action-set disjoint by design, adding `distinctUntilChanged`/debounce where a feedback loop is plausible, and writing effect unit tests with marble diagrams that assert exactly one output emission per input.

6. Interview Questions & Answers

Q1. Why can't you just use a plain `@Injectable` service with public properties for shared state?
A: Because Angular's change detection has no way to know the property changed. A plain property mutation (`this.cartService.items.push(x)`) doesn't emit any notification, so components that aren't the one performing the mutation won't re-render unless they happen to run change detection for an unrelated reason. You need either a reactive primitive (`BehaviorSubject`, `signal`) that consumers subscribe/react to, or manual event plumbing, which doesn't scale.

Q2. What's the minimum viable "state management pattern" you'd reach for before considering a library?
A: A singleton service (`providedIn: 'root'`) that holds a private `signal` (or `BehaviorSubject` if RxJS composition is needed), exposes it read-only via `.asReadonly()` (or `.asObservable()`), and exposes named methods as the only mutation API, always producing new object/array references. This is often sufficient for the majority of apps and is the pattern interviewers expect you to be able to write from memory.

Interviewer intent: This question checks whether the candidate over-indexes on "the library" as the answer rather than understanding the underlying primitives — a candidate who can only recite NgRx APIs but can't build this from scratch usually hasn't internalized *why* NgRx is shaped the way it is.

Q3. What does "unidirectional data flow" mean, and why does it matter?
A: Data flows in one direction around a loop: the view dispatches an intent (an action or a method call), a single, well-defined transition function computes new state from old state plus that intent, and the new state flows back down to re-render the view. The view never mutates state directly. It matters because it makes every state change traceable to exactly one code path, which is what makes debugging, testing, and reasoning about a large app tractable — versus a model where any component can mutate a shared object from anywhere, making bugs require a codebase-wide search.

Q4. Why must reducers (and signal `update()` callbacks) be pure and produce new references instead of mutating?
A: Two reasons. First, Angular's `OnPush` components, `computed()` signals, and NgRx's `createSelector` all use reference equality to detect whether something changed — mutating in place returns the same reference, so these mechanisms conclude nothing changed and skip updates, causing stale UI. Second, purity and immutability are what make time-travel debugging, action replay, and reliable unit testing possible: given the same `(state, action)` pair, a pure reducer always produces the same result, and every past state remains a valid, un-corrupted snapshot.

Q5. Walk through what happens, end to end, when a component dispatches an NgRx action that requires an HTTP call.
A: The component calls `store.dispatch(TodoActions.addRequested({text}))`. The `Store` (backed by a `BehaviorSubject`-like mechanism) synchronously runs the combined root reducer with the current state and this action; the feature reducer for `addRequested` typically flips a `loading` flag and pushes the new immutable state, causing subscribed selectors/components to re-render (e.g., show a spinner). Separately, an `Effects` class has an effect defined with `createEffect(actions$.pipe(ofType(addRequested), mergeMap(...)))` — it receives the same action stream, calls the HTTP API, and on response maps the result to a new action (`addSuccess` or `addFailure`), which it emits back into the actions stream. That action gets dispatched, runs through the reducer again, produces the final state (todo added, loading false), and selectors/components re-render again.

Q6. What is a "selector" in NgRx and how does `createSelector` achieve memoization?
A: A selector is a pure function that derives a piece of view-model data from the state tree, e.g., `selectActiveCount`. `createSelector` wraps a projector function together with its input selectors and caches the last set of input *values* (by reference) alongside the last computed result; on each call it re-runs only the input selectors, and if every input's result is `===` to last time's, it returns the cached result without invoking the (potentially expensive) projector. This means a selector only recomputes when the specific slice(s) of state it depends on actually changed reference, not whenever *any* part of the global state tree changes.

Q7. How is `computed()` in Angular signals similar to `createSelector` in NgRx?
A: Both are pure derivations with automatic memoization based on their dependencies' identity/values. `computed()` tracks exactly which signals were read during its last evaluation and only recomputes when one of those specific signals' values change (compared with the signal's equality function, `Object.is` by default); `createSelector` tracks which "input selectors" it was given and only recomputes when their *results* differ by reference. The philosophical difference: `computed()`'s dependency tracking is automatic (it observes what you read), while `createSelector`'s is explicit (you declare the input selectors up front) — but the memoization contract is the same.

Interviewer intent: Looking for whether the candidate sees NgRx and signals as two different vocabularies for one shared underlying idea (fine-grained reactive memoization), rather than as unrelated technologies.

Q8. When would you choose full NgRx over a signal-based service, even in 2026 with modern Angular signals available?
A: When the app has genuinely complex, cross-cutting asynchronous workflows — chains of side effects with retry/cancellation/race-condition handling that benefit from RxJS operators (`switchMap`, `debounceTime`, `exhaustMap`); when a large team needs an enforced, consistent convention across many state slices so any engineer can jump into any feature's state code and recognize the shape; or when you have a hard requirement for devtools-based time-travel debugging / action-level audit logging (e.g., regulated domains where "what sequence of actions led to this state" must be reconstructable). For most CRUD-shaped line-of-business features, a signal-based store (plain or NgRx Signal Store) is sufficient and simpler.

Q9. What is NgRx Signal Store and how does it differ from classic NgRx Store?
A: NgRx Signal Store (`@ngrx/signals`) is a DI-based, signal-native state container built with `signalStore()` and composable "features" — `withState` (initial state), `withComputed` (derived signals), `withMethods` (the mutation/side-effect API, using `patchState` for guarded immutable updates), `withHooks` (lifecycle), and `withEntities` (normalized collection helpers). Unlike classic NgRx, there are no Actions, Reducers, or Effects classes — mutation happens via direct method calls that internally call `patchState`, and side effects are just methods using `async`/`await` or `rxMethod`. It keeps the DI-scoping, structure, and entity conventions people like about NgRx while removing the action/reducer/effect ceremony, positioning it between a hand-rolled signal service and full NgRx.

Q10. What's the danger of directly mutating an array inside an NgRx reducer or a signal `update()` callback, even if the reducer function itself is otherwise "pure" in structure?
A: If the reducer does `state.todos.push(newTodo); return {...state}` — spreading the *outer* object but mutating the *inner* array in place — the outer state object is technically new, but `state.todos` is still the same array reference as before. Any selector memoized on `state.todos` (like `createSelector(selectTodos, ...)`) will see no reference change on that slice and return a stale cached result, even though the returned top-level state object differs. This is a classic partial-immutability bug: immutability must be maintained at every level of the object graph that a selector/computed depends on, not just at the top level.

Q11. How would you debug a component that isn't re-rendering after you dispatch an action / call a store method, when you've confirmed (via devtools/console.log) that the underlying data did change?
A: First suspect a broken reference — check whether the reducer/update callback mutated in place instead of returning a new object/array at every level the component's selector/computed reads. Second, check `OnPush` components: confirm the input/signal binding is actually the one being read in the template (not a stale local copy assigned once in `ngOnInit`). Third, for NgRx, check whether a custom selector uses a non-memoized inline function recreated on every call (breaking `distinctUntilChanged`'s reference comparison) versus a properly memoized `createSelector`. Fourth, enable NgRx's `runtimeChecks: { strictStateImmutability: true, strictActionImmutability: true }` in development, which throws if you attempt to mutate state or actions directly, surfacing the bug immediately instead of silently.

Interviewer intent: This is a practical debugging question meant to distinguish candidates who understand memoization/reference-equality mechanics deeply enough to diagnose a real production symptom, versus those who only know the happy-path API.

Q12. What is Akita/Elf, and why would/wouldn't you pick them over NgRx?
A: Akita and Elf are both alternative, lighter-weight state management libraries for Angular (Elf is the spiritual successor to Akita, from the same core contributors, designed to be more modular/tree-shakable and framework-agnostic). Both use a more object-oriented/imperative store API (`store.update(state => ({...}))`) instead of the Action/Reducer ceremony, while still providing structured conventions (entity stores, query objects, devtools integration) that a raw `BehaviorSubject` service lacks. You'd pick one of them over NgRx when you want NgRx-like structure and entity helpers but find the Actions/Reducers/Effects boilerplate excessive for your team's needs; you'd stick with NgRx when you want the largest ecosystem, most job-market-recognized pattern, and strict enforced unidirectional flow via actions.

Q13. What is an "effect causing an infinite dispatch loop," and how do you prevent it structurally?
A: It happens when an NgRx effect's listened-to action set and its dispatched action set overlap such that a dispatched action eventually re-triggers the same (or another) effect that dispatches it again, forming a cycle — e.g., an effect listening for `[A, B]` that dispatches `A` when handling `B`, and another effect listening for `A` that dispatches `B` on success. This causes runaway CPU/memory usage and, in production, an unresponsive app. Prevent it structurally by designing each effect's input action types and output action types as disjoint sets by convention/lint rule, code-reviewing any effect whose dispatched action could plausibly satisfy its own `ofType` filter (directly or transitively), and writing marble-diagram unit tests asserting exactly one output per input rather than relying on manual tracing.

Q14. How does immutability specifically enable "time-travel debugging" in NgRx DevTools?
A: Because every dispatched action produces a brand-new, complete state snapshot (rather than mutating the previous one), NgRx DevTools can retain every historical state object in memory as an immutable, independent record, tagged with the action that produced it. "Time travel" is then just re-rendering the app UI using an *older* retained state object — which only works because that older object was never subsequently mutated by anything downstream. If reducers mutated state in place, "past" snapshots would silently reflect *current* data (since they're the same object references), making time travel meaningless.

7. Quick Revision Cheat Sheet

- **State management problem = single source of truth + defined write path + defined read/notify path.** Every pattern below is just a different implementation of these three.
- **Ladder (climb only as needed):** local component state → plain shared service →

service+`BehaviorSubject` → service+`signal` → Akita/Elf/hand-rolled signal store → NgRx Signal Store → full NgRx.

- **Service + signal**: `private _x = signal(...)`, expose `.asReadonly()`, mutate via named methods with `update()` / `set()` producing new references; `computed()` for derived state (auto-memoized).
- **NgRx core pieces:** Actions (typed "what happened" events) → Reducers (pure `(state, action) => newState`) → Store (holds the tree, backed by a BehaviorSubject-like push mechanism) → Selectors (`createSelector`, memoized by reference-equality of inputs) → Effects (isolate side effects, listen to actions via `ofType`, dispatch new actions).
- **NgRx Signal Store:** `signalStore()` + `withState` / `withComputed` / `withMethods` / `withHooks` / `withEntities`; mutate via `patchState`; no Actions/Reducers/Effects ceremony.
- **Unidirectional flow:** view → intent (action/method call) → single transition function → new state → view re-render. Never view → direct state mutation.
- **Immutability is mandatory** because every memoization mechanism (`OnPush`, `computed()`, `createSelector`, `distinctUntilChanged`) relies on reference equality, not deep equality — mutate in place and downstream consumers silently miss the update.
- **Decision rule:** scope of sharing + complexity of async orchestration + team size + need for devtools/audit trail determines the rung; don't default to NgRx, don't refuse structure at scale.
- **Common bugs:** in-place mutation (stale reference → skipped re-render), non-memoized projector functions creating new references every call, effects whose input/output action sets overlap (infinite dispatch loop), over-engineering trivial state with full NgRx.
- **Akita/Elf:** NgRx-adjacent structure (entity stores, devtools) without Action/Reducer/Effect ceremony; Elf is the modern, more modular successor to Akita.

Created By - Durgesh Singh